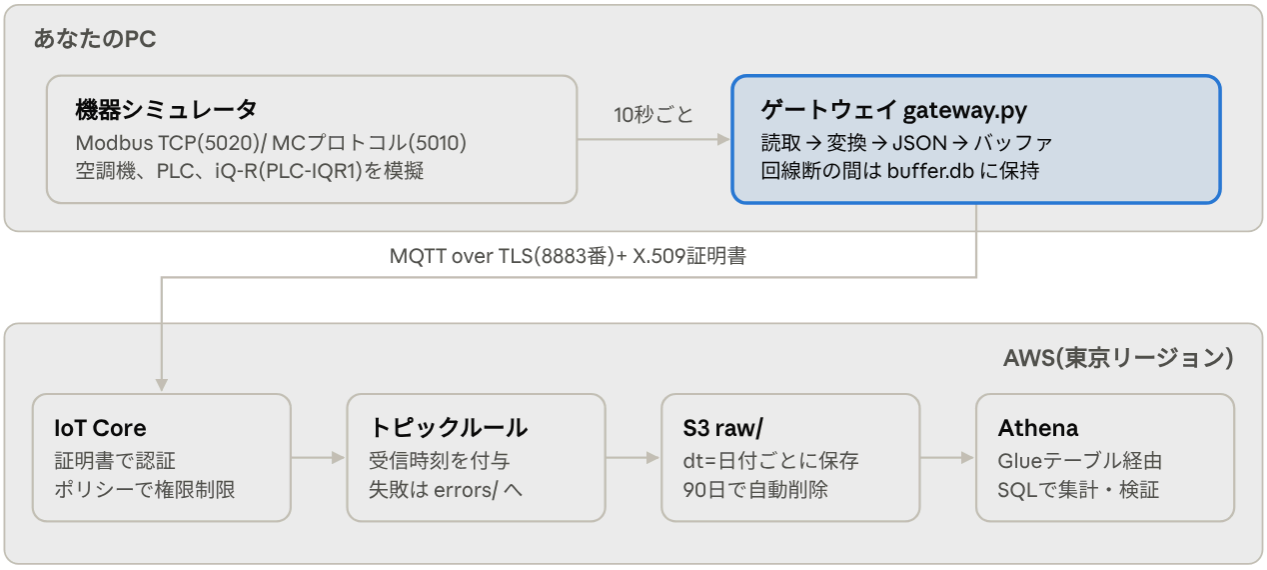
IoT学習ハンズオン:PC+シミュレータ+AWS

Oct 10, 2026 · @Reo Kojima

全体像

PC1台の上で「空調機とPLC」を模擬し、自作ゲートウェイでModbus読み取り→MQTT送信し、AWS IoT Core経由でS3に貯めてAthenaのSQLで分析するところまでを通します。所要時間は準備込みで半日〜1日、費用はAWS新規アカウントの無料プランならクレジット内に収まり、既存アカウントでも学習規模なら月数十円〜数百円です(時間課金のサービスは作りません)。

機器の値はゲートウェイを経てAWSでSQL検索できる形になる



学習キットの構成 · PC側2つ、AWS側4つ

PC側は自分で書いたコードが動く範囲、AWS側はCloudFormationで一括作成・一括削除できる範囲です。

使うのは学習キット iot-lab.zip です。中身は次のとおりです。

ファイル	役割	学べること
simulator/device_simulator.py(Modbus)、mc_plc_simulator.py(iQ-R)	空調機とPLCを模擬するModbus TCPサーバーと、iQ-Rを模擬するMCプロトコル(3Eフレーム)サーバー	レジスタとデバイス、スケーリング、ワード順

ファイル	役割	学べること
gateway/points.json	ポイントリスト(タグ表)	現場で最初に入手すべき資料の形
gateway/gateway.py	収集→変換→バッファ→MQTT送信	ストア&フォワード、UTC、連番、QoS1
aws/template.yaml	IoTポリシー、ルール、S3、Glueテーブル、Athena	最小権限、ルールエンジン、パーティション
aws/setup_aws.py / teardown_aws.py	AWS環境の作成・削除、証明書発行	X.509証明書による機器認証、後片付け
aws/queries.sql	Athenaのサンプルクエリ	時系列集計、遅延・欠損・重複の確認

準備(30〜60分)

Python、AWSアカウント、AWS CLIの3つと、予算アラートを用意します。Windows・Mac・Linuxどれでも同じ手順で動きます。以下のコマンドはWindowsのPowerShell向けです(ターミナルを2つ使うときはPowerShellの窓を2つ開きます)。

1. **Python 3.10以上**をpython.orgのインストーラで入れます。最初の画面で「Add python.exe to PATH」にチェックを入れてください。PowerShellで `python --version` と打ってバージョンが出ればOKです(Microsoft Storeが開く場合はPATH設定漏れなので、インストーラをやり直します)。
2. iot-lab.zipを右クリック→「すべて展開」し、そのフォルダでPowerShellを開いて(フォルダの空白部分をShift+右クリック→「PowerShellウィンドウをここで開く」、またはエクスプローラーのアドレス欄に `powershell` と入力)、仮想環境を作ります。

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

`Activate.ps1` で「スクリプトの実行が無効」というエラーが出たら、先に次の1行を実行してから再度有効化します。

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

行頭に (.venv) と表示されていれば有効です。新しいPowerShellの窓を開いたときは、毎回 `.venv\Scripts\Activate.ps1` を実行してください。

3. **AWSアカウント**を用意します。ルートユーザーにはMFAを設定し、普段の作業は IAM Identity Center かIAMユーザーで行います。
4. **AWS CLI v2**をWindows用インストーラ(.msi)で入れ、PowerShellを開き直してから `aws configure` (アクセスキー)または `aws configure sso` で認証を設定します。`aws sts get-caller-identity` でアカウントIDが表示されればOKです。
5. **予算アラート**:Billing and Cost Management の Budgets で月額 5 USD 程度の予算を作り、メール通知を設定します。課金の事故を防ぐ一番効く対策です。
6. 初回にシミュレータを起動したとき「Windowsセキュリティの重要な警告」(ファイアウォール)が出たら、「プライベートネットワーク」だけ許可します。このキットはPC内(127.0.0.1)で通信するので、許可しなくても動作します。

リージョンは東京(ap-northeast-1)を前提にしています。変える場合は setup/teardown に `--region` を付けます。

ステップ1:機器を読む(Modbus)

ゴールは、生のレジスタ値(16bitの数字の並び)を「室温27.2℃」のような意味のある値に直せるようになることです。AWSはまだ使いません。

1. ターミナル1でシミュレータを起動します(ポート5020で待ち受け、1秒ごとに値が変化)。

```
python simulator/device_simulator.py
```

2. ターミナル2で、ゲートウェイを「画面表示のみ・生値表示あり」で1回動かします。

```
python gateway/gateway.py --console --dump-raw --once
```

3. 次のような出力を見比べます(実際に動かした結果の抜粋)。

```
[raw] AHU-01     addr=3    0x0110          -> room_temp_c=27.2
[raw] PLC-LINE1  addr=100  0x0009 0x0000   -> production_count=9
[raw] PLC-LINE1  addr=102  0x4140 0xFC76   -> motor_current_a=12.062
```

読み解きのポイントは3つです。

- **スケーリング**:0x0110 = 272。ポイントリストの scale 0.1 を掛けて27.2℃。小数を整数×10で持つのは空調・PLCで非常によくある形です。
- **ワード順**:生産カウンタは 0x0009, 0x0000 の順。下位ワードが先なので値は9。逆に読むと 9×65536 = 589,824 という大きな値になります。
- **浮動小数点**:モータ電流は2ワードでIEEE754の32bit浮動小数点。0x4140FC76 ≒ 12.06A。ワード順を逆にすると全く違う数字になります。

やってみる:gateway/points.json の production_count の type を `u32_hw` に変えて再実行し、値が化けることを確認してから元に戻してください。実機で「値がおかしい」ときの最初の疑いどころです。

ステップ1b:三菱電機 iQ-R をMCプロトコルで読む

iQ-RのPLCは、Modbusではなく三菱独自のMCプロトコル(SLMP)で読むのが基本です。キットの mc_plc_simulator.py がiQ-R役になり、ゲートウェイは機器ごとにModbusとMCプロトコルを切り替えて読みます。空調機はModbus、PLCはMCプロトコルという、現場によくある混在構成です。

1. PowerShellをもう1つ開き(仮想環境を有効化して)、iQ-R役のシミュレータを起動します。ポート5010で待ち受けます。

   ```
   python simulator/mc_plc_simulator.py --verbose
   ```

2. ステップ1と同じコマンドでゲートウェイを動かすと、PLC-IQR1 の行が増えています(実際に動かした結果の抜粋)。

   ```
   [raw] PLC-IQR1   D100   0x0057 0x0000  -> production_count=87
   [raw] PLC-IQR1   D102   0x007A         -> conveyor_speed_mpm=12.2
   [raw] PLC-IQR1   D104   0xAE17 0x40F9  -> motor_current_a=7.803
   [raw] PLC-IQR1   M0     1             -> line_running=1
   ```

   シミュレータ側には、ゲートウェイがどのデバイスを読みに来たかが表示されます。

シミュレータのデバイス割り付けは次のとおりです。実務では、PLCプログラマからこういう「デバイス表」を受け取ることが出発点になります。

デバイス	内容	形式	points.json の type
D100-D101	生産カウンタ	32bit整数。D100が下位ワード	u32_lw
D102	搬送速度	10倍した整数(125→12.5 m/min)	u16, scale 0.1
D104-D105	モータ電流	単精度実数。D104が下位ワード	f32_lw
D110	異常コード	0=正常	u16
M0	ライン稼働中	1ビット	bit
M1	異常発生中	1ビット	bit

Modbusとの違いで押さえるのは次の点です。

- **アドレスはデバイス名**:points.json の address に「D100」「M0」のように書きます。X、Y、Wの番号は16進数(X10は16番目)なので注意します。

- **三菱は下位ワードが先**:2ワードの値はD100が下位、D101が上位が基本です。ステップ1のModbus版PLC(PLC-LINE1)の実数は上位ワード先だったので、同じ「モータ電流」でも機器によって並びが逆になることが分かります。
- **通信形式を合わせる**:3Eフレーム、バイナリ、TCPで通信しています。PLC側の設定(交信データコード)がASCIIだと読めません。

実機のiQ-Rで読むときのPLC側の設定(GX Works3)

実機では、PLC側でMCプロトコルの受け口を開けておく必要があります。PLCの設定変更になるので、必ずPLC担当者と相談して進めてください。

1. ナビゲーションの「パラメータ」→CPUユニット→「ユニットパラメータ」を開き、内蔵Ethernetポートの「基本設定」でIPアドレスを確認・設定します。
2. 「交信データコード」を「バイナリ」にします。
3. 「外部機器接続構成」の詳細設定を開き、右側の一覧から「SLMP接続機器」を左の表にドラッグします。プロトコルを TCP にし、ポート番号を既存の設定と重ならない番号にします。1つのSLMP接続機器につき接続は1つなので、ゲートウェイを複数台つなぐ場合はその数だけ追加します。
4. 同じ画面にある、RUN中の書き込み(オンライン変更)を許可するかどうかの設定(英語版の表記は Enable/Disable Online Change)は、外部からの書き込みに関わります。読み取り専用で使うなら、書き込みを許可するかどうかをPLC担当者と決めておきます。
5. 設定を反映して閉じ、パラメータをPLCに書き込み、PLCをリセット(または電源を入れ直し)します。
6. gateway_config.json の mc に、PLCのIPアドレスとポート番号を書きます。

設定の詳細は、[iQ-R内蔵Ethernetの設定例(deviceWISE)]や、三菱電機の「MELSEC iQ-R Ethernetユーザーズマニュアル」「SLMPリファレンスマニュアル」で確認してください。

ステップ2:ゲートウェイの中身を理解する

gateway.py は「読む→変換→1台1メッセージのJSONにする→バッファに積む→送れたら消す」を10秒ごとに繰り返します。送られるメッセージは次の形です。

```
topic: lab/gw01/AHU-01/telemetry
{"message_id": "b210cdde-…", "gateway_id": "gw01", "equipment_id": "AHU-01",
"kind": "hvac",
 "ts": "2026-10-10T10:39:24.711Z", "seq": 1791628765, "status": "ok",
 "metrics": {"run_status": 1, "mode": 1, "setpoint_c": 26.0, "room_temp_c":
27.2, "humidity_pct": 55.4, "alarm_code": 0}}
```

各項目には実務上の理由があります。

項目	なぜ必要か
ポイントリスト(points.json)	アドレス・型・スケールをコードから分離。機器が増えてもコードを変えずに済む
ts(UTC、末尾Z)	日本時間で保存すると夏時間のある国や他システムとの結合で破綻する。表示時にJSTへ変換する
seq(連番)	欠損(飛び)の検出用
message_id	QoS1は「最低1回」なので重複があり得る。DB側で重複排除する鍵
status	通信エラーも「comm_error」として送る。データが無い理由が後から分かる
トピック lab/ゲートウェイ/機器/telemetry	IoTポリシーで「自分のトピックにだけ送れる」制限や、ルールでの振り分けに使う
SQLiteバッファ	回線断の間は buffer.db に貯め、復旧後に古い順に再送。PUBACKが返った分だけ消す

やってみる: `python gateway/gateway.py --console` を動かしたままシミュレータを Ctrl+C で止め、status が comm_error に変わること、再起動すると ok に戻ることを確認します。

ステップ3:AWS IoT Coreへ送る

setup_aws.py を1回実行すると、AWS側の受け皿づくりと証明書の発行、ゲートウェイ設定ファイルの作成までが自動で終わります。

1. AWS環境を作ります(1〜3分)。

   ```
   python aws/setup_aws.py
   ```

   中では次のことが起きています。

   - CloudFormationで、S3バケット、IoTのモノ(gw01)とポリシー、トピックルール、Glueテーブル、Athenaワークグループを作成
   - X.509証明書と秘密鍵を発行し、gateway/certs/ に保存。証明書をポリシーとモノに紐づけ
   - IoT Coreの接続先(エンドポイント)を調べ、gateway/gateway_config.json を作成
2. AWSコンソールで **AWS IoT → MQTTテストクライアント** を開き、`lab/#` をサブスクライブします。
3. ゲートウェイを送信モードで起動します。

```
python gateway/gateway.py
```

ログに「MQTT接続: Success」「送信 2 件 / バッファ残 0 件」が出て、テストクライアントにJSONが届けば成功です。

送信中は、シミュレータ2つ(device_simulator.py と mc_plc_simulator.py)も起動しておきます。片方だけだと、もう片方の機器は comm_error として送られます。以前のキットで setup_aws.py を実行済みで、起動時に「設定ファイルに mc の接続先がありません」と出た場合は、setup_aws.py をもう一度実行すれば設定が作り直されます(証明書はそのまま使われます)。

ここで押さえる知識は次のとおりです。

- **証明書認証**:ID/パスワードではなく、機器ごとの証明書でTLS接続します。秘密鍵(private.pem.key)が漏れるとなりすましされるので、Gitや共有フォルダに置きません。
- **最小権限のポリシー**:gw01は「client ID が gw01 のときだけ接続できる」「lab/gw01/ 以下にだけ送れる」。試しに gateway_config.json の topic_prefix を変えると、送信が拒否され切断されるのが分かります。
- **ルールエンジン**: `SELECT *, timestamp() AS received_at_ms FROM 'lab/+/+/telemetry'` で全機器のメッセージを拾い、AWSで受け取った時刻を足してS3に1件1ファイルで保存します。保存に失敗したものは errors/ に退避されます(デッドレターの考え方)。

ステップ4:S3とAthenaで分析する

データはS3の `raw/dt=YYYY-MM-DD/` に溜まり、AthenaのSQLでそのまま検索できます。サーバーを立てない「データレイク」型の構成です。

1. S3コンソールで、setup_aws.py が表示したバケットの raw/ を開き、日付フォルダとJSONファイルができていることを確認します。
2. Athenaコンソールを開き、ワークグループを **iot-lab**、データベースを **iot_lab** に切り替えます(結果の保存先は設定済み)。
3. aws/queries.sql のクエリを上から順に実行します。

クエリ	見るもの
1. 届いたデータ	生データの形
2. 室温の時系列	map型の metrics から値を取り出す方法
3. 10分平均	ダウンサンプリング(長期保存・グラフ化の基本)
4. 送信遅延	ts(読んだ時刻)と received_at_ms(届いた時刻)の差

クエリ	見るもの
5. 欠損	seq の飛び
6. 重複	同じ message_id の重複
7. エラー集計	comm_error と異常コードの件数

注意点が2つあります。

- **dt はUTCの日付**です。日本時間の0時〜9時のデータは前日の dt に入ります。
- Athenaは**読んだデータ量で課金**されます(ワークグループで1クエリ1GBまでに制限済み)。必ず `WHERE dt = …` で日付を絞る習慣をつけてください。学習規模ならほぼ0円です。

ステップ5:障害を起こして学ぶ

実務で差がつくのは「うまくいかないとき」の設計です。わざと障害を起こし、データにどう表れるかをAthenaで確かめます。

実験	やり方	期待される結果	確認クエリ
回線断	ゲートウェイ起動中にPCのWi-Fiを1〜2分切り、戻す	ログで「バッファ残」が増え、復旧後にまとめて送信。データは欠けない	4(遅延が大きい行が出る)
機器停止	シミュレータを Ctrl+C で止め、1分後に再起動	その間 status = comm_error のメッセージが届く	7
ゲートウェイ停止	gateway.py を止めて1分後に再起動	その間のデータは存在しない(バッファにも無い)	5(seq の飛び)
重複	送信直後に回線を切るなどPUBACKを取りこぼす	同じ message_id が2件届くことがある	6
権限外送信	topic_prefix を変えて送る	接続が切られ、データは届かない	なし(ログで確認)

ゲートウェイ停止時の欠損は、このキットでは防げません。実務ではゲートウェイ自体の死活監視(一定時間データが来なければアラート)で検知します。重複は、DBに入れる段階でmessage_id(または機器ID+時刻)を主キーにしたUPSERTで吸収するのが定石です。

実験のたびに「どの障害が、どのクエリで、どう見えるか」をメモしておくと、そのまま監視設計の叩き台になります。

片付け・トラブル対応・次のステップ

学習が一区切りしたら、次のコマンドでAWS側をすべて削除します。証明書の無効化・削除、S3の中身の削除、スタック削除、ローカルの証明書削除まで自動で行います。再開するときは setup_aws.py をもう一度実行するだけです。

```
python aws/teardown_aws.py
```

作るものはすべて従量課金で、起動しっぱなしで課金されるサービス(RDS、Timestream for InfluxDB、NATゲートウェイなど)は含みません。費用の大半はIoT Coreのメッセージ数なので、送信周期を10秒より短くしないのが節約のコツです。

よくあるつまずき

症状	原因と対処
シミュレータ起動で Address already in use	5020番ポートが使用中。`--port 5021` で起動し、gateway_config の modbus.port も合わせる
ゲートウェイが接続と切断を繰り返す	client_id や topic がポリシーと不一致、または証明書が未アタッチ。setup_aws.py を再実行
MQTT接続でSSLエラー	社内プロキシやセキュリティソフトが8883番ポートを遮断。自宅回線で試す
Athenaで0件	dt がUTCの日付か確認。S3の raw/ にファイルがあるか確認
S3に何も来ない	errors/ にファイルがあればルールの保存失敗。IoT Coreのルール画面でエラーを確認
setup_aws.py で権限エラー	作業ユーザーにCloudFormation・IoT・IAM・S3・Glue・Athenaの権限が必要

次のステップ

- [ ] Grafana(ローカルのDocker)とAthenaをつないでグラフ化する
- [ ] BACnetシミュレータを追加し、空調機をBACnet/IPで読む
- [ ] OPC UAのサンプルサーバーを立て、PLC役をOPC UAに置き換える
- [ ] IoTルールにLambdaを追加し、機器ごとの最新値をDynamoDBに保存する
- [ ] ゲートウェイをAWS IoT Greengrassのコンポーネントにする